

OPTIMUM RACE

Agent build prompts - Steps 2 to 13

Copy one prompt at a time into your coding agent. Test each step on your phone before sending the next.

How to use this

- Step 1 is your existing prompt. Use it exactly as written, then continue here at Step 2.
- Send ONE prompt at a time. Wait for the agent to finish and test it before moving on.
- If something breaks, use the fix prompt on the last page instead of moving forward.
- Every prompt repeats the same guard rails so the agent does not jump ahead.

Color guide

Phase A Steps 2-8	Core racing: driving, boost, race system, track, AI, difficulty, tracks
Phase B Steps 9-11	Progression: profile, XP, unlocks
Phase C Steps 12-13	Events, daily rotation and final polish

What is deliberately left out

Multiplayer, wallets, blockchain, tokens, online leaderboards and live Optimum network data are not built. A game that shows fake network numbers or asks for a wallet would look like an official product. The Optimum theme comes from names, tracks and verified facts only.

Build Step 2 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 2.
- Steps 1 to 1 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and new boost rules, laps, AI, new tracks.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Improve how the car feels to drive. Nothing else.

STEP 2 REQUIREMENTS:

1. Make steering scale with speed: tighter at low speed, stable at high speed, and no steering while stationary.
2. Add grip and light drift: the car's velocity gradually aligns with its facing direction, with a small sideways slide when turning hard at high speed.
3. Off-road behavior: driving on grass outside the road slows the car strongly. The edge of the playable area gives a soft bounce instead of a hard stop.
4. Add a smooth follow camera with slight look-ahead in the direction of travel.
5. Improve the car visuals: clear car shape, headlights, rotation, and simple tire marks when drifting or braking hard (cap the number of marks for performance).
6. Add optional engine sound generated with Web Audio (no audio files). Sound is off by default with a visible mute toggle.
7. Put every tuning value (acceleration, max speed, friction, turn rate, grip) in one config object at the top of a config file.

TESTING: Test and report on each of these:

- Steering, acceleration and braking feel responsive on desktop and on a phone
- The car never gets stuck on the boundary
- Editing config values visibly changes handling
- Smooth performance on a mid-range phone
- Everything from Step 1 still works

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 2.

Build Step 3 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 3.
- Steps 1 to 2 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and laps, AI, upgrades, XP, new tracks.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Turn the simple boost into a meter-based mechanic. Theme name in the UI: CODED BOOST.

STEP 3 REQUIREMENTS:

1. Add a boost meter from 0 to 100 in the HUD.
2. The meter charges slowly while driving on the road at high speed, and by collecting shard pickups placed on the track (small cyan diamonds, +20 each).
3. Boost activates with the BOOST button, Space or Shift. It lasts about 1.5 seconds, drains the meter, and gives a speed multiplier plus a higher temporary max speed. It cannot start below 25 percent.
4. Add a 3 second cooldown after each use with a visible indicator.
5. Effects: speed lines or trail, a subtle camera zoom, and a HUD glow while boosting. Respect prefers-reduced-motion.
6. Track boosts used and peak boosted speed. Show both on the finish screen.
7. Shard pickups respawn on Race Again. All boost tuning values live in the config file.

TESTING: Test and report on each of these:

- Meter charges on the road and from pickups
- Boost cannot start below 25 percent
- Cooldown works and shows clearly
- BOOST touch button works on mobile
- Race Again resets meter, pickups and stats

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 3.

Build Step 4 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 4.
- Steps 1 to 3 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and AI opponents, positions against other cars, XP, new tracks.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Turn free driving into a structured race.

STEP 4 REQUIREMENTS:

1. Create a race state machine with the states: MENU, COUNTDOWN, RACING, PAUSED, FINISHED.
2. Add a 3-2-1-GO countdown. Player input is locked until GO.
3. The race is 3 laps. A lap only counts after passing the checkpoints in order, so shortcuts cannot skip a lap. Show LAP 2/3 in the HUD.
4. Record every lap time, show the current lap timer, and highlight the best lap.
5. Add a Pause button plus Escape or P. The pause menu has Resume, Restart and Quit to menu.
6. The finish screen shows total time, lap splits, best lap, maximum speed and boosts used, with RACE AGAIN and MENU buttons.
7. Restarting fully resets all race state.

TESTING: Test and report on each of these:

- Countdown locks input
- Checkpoints prevent shortcuts
- Lap counter and splits are correct
- Pause, resume, restart and quit work
- Race Again gives a clean race
- Mobile and desktop controls

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step.

Complete and test ONLY Step 4.

Build Step 5 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 5.
- Steps 1 to 4 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and extra tracks, AI, weather.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Replace the simple track with a data-driven track and renderer. First track name: FLEXNODE CIRCUIT.

STEP 5 REQUIREMENTS:

1. Define a track as data: centerline points (closed loop), road width, start/finish position and checkpoint positions. Keep track data separate from game logic.
2. Write a renderer that draws the road from that data: smooth curves, edge lines, kerbs on corners, a start/finish line and subtle checkpoint gates.
3. Provide reusable helper functions `isOnRoad(x, y)` and `getNearestTrackPoint(x, y)` that future AI can use.
4. Add cheap scenery: grass, trees or pylons, and small glowing node markers. Pre-render static scenery to an offscreen canvas.
5. Add a HUD minimap showing the track outline and the player dot.
6. Flexnode Circuit has at least 8 turns, one hairpin and one long straight.
7. Keep world boundaries with walls or a soft bounce.

TESTING: Test and report on each of these:

- Track renders correctly on small and large screens
- `isOnRoad` is accurate at edges and corners
- Minimap tracks the player
- Checkpoints and laps still work on the new track
- Frame rate stays smooth

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 5.

Build Step 6 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 6.
- Steps 1 to 5 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and difficulty selection, multiple tracks, online features.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add computer-controlled rivals. Rivals are named STANDARD GOSSIP 1 to 3.

STEP 6 REQUIREMENTS:

1. Add 3 rival cars in colors clearly different from the player.
2. Rivals follow the track centerline with small lane offsets, and slow down for sharp upcoming corners.
3. Each rival has a slightly different top speed and skill, consistent within a race.
4. Add a gameplay stall: each rival randomly slows for about 0.5 to 1 second every few seconds, with a small flash icon. This is a gameplay mechanic only and must not claim to represent real network behavior.
5. Add simple car-to-car collision: push apart with a small speed loss.
6. Show live position (POS 2/4) and a small standings list in the HUD.
7. Use a staggered grid start behind the start line.
8. The finish screen shows final standings and the player's place.
9. AI must use only the track helper functions from Step 5.

TESTING: Test and report on each of these:

- Rivals finish laps without getting stuck
- Stalls are visible and fair
- Collisions do not trap cars
- Position display updates correctly
- Standings are correct at the finish
- Performance with 4 cars on a phone

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 6.

Build Step 7 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 7.
- Steps 1 to 6 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and multiple tracks, profile, XP.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add selectable difficulty for the rivals.

STEP 7 REQUIREMENTS:

1. Add EASY, NORMAL and HARD selection before the race.
2. Put difficulty settings in a config object: rival top speed multiplier, stall frequency and duration, corner braking skill, rubber banding strength.
3. Add light rubber banding: rivals ease off when far ahead and push when far behind, capped so it never feels unfair.
4. Save the selection in localStorage (wrapped in try/catch) and preselect it next time.
5. Show the difficulty in the HUD and on the finish screen.
6. Balance target: a decent player wins Easy most of the time, Normal about half the time, and Hard rarely but possibly.
7. Store best race time per difficulty and show 'Best on Normal' style text on the finish screen.

TESTING: Test and report on each of these:

- Each difficulty feels different
- Selection persists after reload
- Works when localStorage is blocked
- Best time stored per difficulty
- Difficulty label appears in HUD and results

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 7.

Build Step 8 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 8.
- Steps 1 to 7 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and profile, XP, unlocks (all tracks are available in this step).
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add two more tracks using the same data format.

STEP 8 REQUIREMENTS:

1. Keep Flexnode Circuit and add MESH HIGHWAY (fast, wide, long straights) and SHARD SPEEDWAY (tight and technical).
2. All tracks use the Step 5 data format. No per-track hacks in game code.
3. Add a track select screen with name, a small preview drawn from the centerline, length and a difficulty rating.
4. Store best time and best lap per track and per difficulty.
5. Shard pickups, checkpoints and lap count are defined per track in data.
6. Race Again keeps the same track. Menu returns to track select.

TESTING: Test and report on each of these:

- All three tracks are completable with rivals
- Track preview matches the real track
- Best times save per track and difficulty
- No regressions in laps, checkpoints or AI on any track
- Track select works on mobile

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 8.

Build Step 9 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 9.
- Steps 1 to 8 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and XP, levels, unlocks, accounts, online features.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add a local player profile and a proper save system.

STEP 9 REQUIREMENTS:

1. Create a storage module with one versioned save object and migrations. Every read and write is wrapped in try/catch.
2. Add a profile screen with player name (max 16 characters, sanitized, default 'Racer') and a car color choice from 6 colors.
3. Track stats: races played, wins, podiums, total race time and best time per track.
4. Update stats automatically after each race.
5. Add Export save (copy a Base64 string) and Import save with a confirmation prompt.
6. Add Reset progress with a confirmation prompt.
7. Use the player name and car color in the HUD and finish screen.
8. Preserve existing best times from earlier steps.

TESTING: Test and report on each of these:

- Profile saves and reloads correctly
- Export then import restores everything
- Reset clears progress
- Game still runs if storage is unavailable
- Invalid or corrupt saves do not crash the game

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step.

Complete and test ONLY Step 9.

Build Step 10 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 10.
- Steps 1 to 9 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and unlocks, events, real-world rewards, tokens.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add XP and levels earned by racing.

STEP 10 REQUIREMENTS:

1. Award XP after each finished race: finishing +50, place bonus (1st +100, 2nd +60, 3rd +30), +20 per clean lap (no wall hits), and a small capped bonus for boost use.
2. Apply a difficulty multiplier: Easy x0.8, Normal x1, Hard x1.3.
3. Use a level curve where level N needs about $100 * N^{1.5}$ XP, defined in config.
4. The results screen shows an XP breakdown line by line and an animated progress bar.
5. Show the level and progress bar on the main menu and profile screen.
6. Show a level-up toast.
7. Save XP and level through the storage module with a migration.
8. Quitting a race early gives no XP.

TESTING: Test and report on each of these:

- XP math matches the breakdown shown
- Level-up triggers once
- Multipliers apply correctly
- XP persists after reload
- Old saves migrate without losing data

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 10.

Build Step 11 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 11.
- Steps 1 to 10 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and purchases, in-game currency, wallets, NFTs.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add cars and tracks that unlock as the player levels up.

STEP 11 REQUIREMENTS:

1. Add 4 cars with different stats (acceleration, top speed, handling, boost capacity), balanced so none is strictly best. Suggested names: Relay, Validator, Shard, Flexnode.
2. Add a car select screen with stat bars and a preview.
3. Unlock by level. Example: car 2 at level 3, car 3 at level 6, car 4 at level 10, Mesh Highway at level 2, Shard Speedway at level 5. Keep these numbers in config.
4. Locked items show a lock icon and the required level, and cannot be selected.
5. Show an unlock toast when a new item becomes available.
6. Save the selected car and apply its stats through the existing physics config.
7. Add a migration for saves that have no unlock data.

TESTING: Test and report on each of these:

- Locked items cannot be selected or raced
- Unlock triggers at the right level
- Each car handles differently
- Selection persists
- Existing players are not locked out of what they already used

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 11.

Build Step 12 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 12.
- Steps 1 to 11 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and rotating or daily schedules, leaderboards.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Add special challenge events defined as data.

STEP 12 REQUIREMENTS:

1. Define each event as data: id, name, description, objective, modifier and XP reward.
2. Add 4 events: BLOCK RUSH (1 lap, beat a target time), STEADY STREAM (finish without hitting a wall), RIVAL GAUNTLET (win on Hard), SHARD HUNTER (collect N pickups in one race).
3. Add an Events screen listing each event with objective, reward and status (available or completed).
4. Starting an event launches a race with the modifier applied and the objective tracked live in the HUD.
5. On finish, show Event complete or Event failed. Award XP once per completion.
6. Save completion state.
7. Apply modifiers through a small hook system (for example onRaceStart, onUpdate, onFinish) so new events can be added without editing core code.

TESTING: Test and report on each of these:

- Each event can be completed and failed
- XP is awarded only once
- Objective progress updates live
- Modifiers reset after the event
- Normal races are unaffected

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 12.

Build Step 13 of a web game called Optimum Race.

IMPORTANT:

- Work ONLY on Step 13.
- Steps 1 to 12 are already complete and working. Do not unnecessarily rewrite existing files.
- Do not implement: multiplayer, blockchain or wallet features, Optimum network integration, fake or simulated network latency, backend servers, and leaderboards, online features, Optimum network integration.
- Keep the code clean, modular and easy to extend in later steps.

GOAL: Turn events into a daily rotating system and finish the game with polish. This is the final step.

STEP 13 REQUIREMENTS:

1. Build an event engine that features one event per day, chosen deterministically from the local date (no backend). Show Today's event on the menu with a countdown to the next rotation.
2. Give a one-time daily bonus XP for completing the featured event.
3. Support modifiers: stall storm, narrow road, low grip, boost rush. Add 2 new events that use them.
4. Save event history and a completion streak for the last 7 days.
5. Add a pause menu, a sound toggle, a haptics toggle (vibrate on boost and collision, off by default) and reduced-motion support.
6. Show a Did you know fact on the finish screen, rotating through ONLY these verified facts: Optimum runs as a permissionless network of flexnodes that anyone can operate. | mump2p is RLNC-accelerated pub/sub that stays compatible with libp2p and gossipsub. | RLNC stands for Random Linear Network Coding. It mixes message fragments so lost pieces can be recovered. | With coded shards, a receiver only needs a fraction of them to rebuild the full message. | Optimum says mump2p delivers 6-20x faster block propagation for Ethereum validators. | Flexnodes encode, decode and forward coded gossip, and can serve DeRAM and DeROM requests.
7. Add the disclaimer 'Unofficial fan project, not affiliated with Optimum.' on the menu and footer, with a link to getoptimum.xyz.
8. Add a meta description, Open Graph tags, Twitter card tags and a favicon.
9. Write a README covering how to run locally and deploy to Netlify.
10. Run a full regression of Steps 1 to 12 and fix any bugs found. Add no other new features.

TESTING: Test and report on each of these:

- Daily event changes with the date
- Bonus XP is given once per day
- All modifiers work and reset cleanly
- Facts rotate and disclaimer is visible
- Full playthrough of every track and difficulty
- iPhone Safari and Android Chrome
- Landscape and portrait
- Storage blocked or full

Also confirm every earlier step still works. At the end, list the files you changed and how I can test this step. Complete and test ONLY Step 13.

Fix and regression prompts

BUG FIX PROMPT

Fix a bug in Optimum Race. Do NOT add features and do not rewrite unrelated files.

Bug: [describe what happens, on which device and browser]

Expected: [what should happen]

Steps to reproduce: [1, 2, 3]

Find the root cause, apply the smallest safe fix, and explain the cause in two sentences.

Then retest the affected feature and confirm nothing from earlier steps broke.

REGRESSION CHECK PROMPT

Do not add features. Run a regression check of everything built so far in Optimum Race.

Test: menu, start race, countdown, driving, boost and cooldown, laps and checkpoints, pause, finish screen, Race Again, difficulty, all tracks, profile, XP, unlocks and events (whichever exist).

Test desktop keyboard and mobile touch controls, portrait and landscape, and behavior when localStorage is blocked.

Report a pass or fail table and fix only what fails, using the smallest possible changes.

PERFORMANCE PROMPT

Do not add features. Profile Optimum Race for performance on a mid-range phone.

Check frame rate, memory growth over 10 races, number of particles, offscreen canvas usage and event listener cleanup.

Fix only real problems and report before and after results.